

TRABAJO UML

LUIS ALEJANDRO QUIROS ZAPATA

FICHA

3169892

SERVICIO NACIONAL DE APRENDIZAJE SENA

CTMA

ANALISIS Y DESARROLLON DE SOFTWARE

2025

LUISALEJANDRO QUIROS ZAPATA

¿Qué es un dato?

Un **dato** es una representación simbólica de un hecho, concepto o instrucción que por sí solo no tiene significado completo. Puede ser un número, palabra, imagen, fecha, entre otros. Es la unidad mínima de información que, al ser procesada o interpretada, se convierte en **información**.

Ejemplo: "25", "rojo", "2025-07-08".

¿Qué es información?

La **información** es el conjunto de datos procesados y organizados de forma que adquieren un significado útil para quien los recibe. Permite tomar decisiones, resolver problemas o comprender una situación.

Ejemplo: “El cliente tiene 25 años y su color favorito es el rojo”.

¿Cómo se puede garantizar la disponibilidad de los datos en la empresa?

Para garantizar la **disponibilidad** de los datos en una empresa, se deben implementar estrategias como:

- Uso de **copias de seguridad** (backups) periódicas.
- Infraestructura de **alta disponibilidad** (servidores redundantes, balanceo de carga).
- Sistemas de **respaldo en la nube**.
- **Planes de recuperación ante desastres** (DRP).
- **Monitoreo continuo** y mantenimiento de sistemas.
- Control de acceso para evitar interrupciones por errores humanos o amenazas internas.

¿A qué nos referimos con confiabilidad de la información?

La **confiabilidad de la información** se refiere a que los datos sean **consistentes, precisos y veraces**, y que puedan ser utilizados con certeza para tomar decisiones. Una información confiable proviene de fuentes verificadas y no ha sido alterada o manipulada sin autorización.

¿Por qué consideras que la integridad de la información es esencial?

La **integridad de la información** es esencial porque garantiza que los datos no han sido alterados de forma no autorizada. Preservar la integridad evita errores en decisiones, fraudes, pérdidas económicas o daños a la reputación. Además, asegura que la información es coherente desde su origen hasta su uso final.

¿Qué tanto puede impactar una falla de autenticidad en los datos de una organización?

Una **falla de autenticidad** (cuando no se puede verificar el origen o legitimidad de los datos) puede tener **graves consecuencias** como:

- **Pérdida de confianza** de clientes y socios.
- **Fraudes** o suplantaciones de identidad.
- **Errores operativos** o estratégicos por basarse en datos falsos.
- **Multas legales o regulatorias**, si se violan normativas.
- **Daño a la reputación** de la empresa

Decimal	Binario
5050	100111011101 0
4096	100000000000 0
4567	100011100111
3800	11101101000
1098	10001001010
1024	10000000000
2048	100000000000

2512	100111010000
1970	11110110010
680	1010101000
512	1000000000
300	100101100
256	100000000
128	10000000
190	10111110
64	1000000
56	111000
10	1010
780	1100001100
912	1110010000
360	101101000
556	1000101100
428	110101100
990	1111101110
604	1001011100
506	111111010
100	1100100

Consulta:

¿Qué operaciones aritméticas pueden ejecutarse con los sistemas numéricos?

¿Cómo se hacen conversiones de un sistema a otro: binario a octal, octal a decimal, decimal

a hexadecimal, etc.?

¿Qué son: bit, nibble, byte?

¿Qué operaciones aritméticas pueden ejecutarse con los sistemas numéricos?

En todos los sistemas numéricos (binario, octal, decimal, hexadecimal) se pueden realizar las **mismas operaciones aritméticas básicas**:

- **Suma**
- **Resta**
- **Multiplicación**
- **División**

La diferencia está en la **base del sistema**. Por ejemplo, en binario (base 2), solo se usan 0 y 1, mientras que en decimal se usan del 0 al 9.

¿Cómo se hacen conversiones entre sistemas numéricos?

1. Binario a Octal

Agrupar los bits en tríos desde la derecha y convierte cada grupo a su equivalente octal.

Ej: $1011110 \rightarrow 000\ 101\ 110 \rightarrow 2\ 7\ 6 \rightarrow 276_8$

2. Octal a Decimal

Multiplica cada dígito por 8^n según su posición (de derecha a izquierda).

Ej: $276_8 \rightarrow (2 \times 8^2) + (7 \times 8^1) + (6 \times 8^0) = 128 + 56 + 6 = 190_{10}$

3. Decimal a Hexadecimal

Divide entre 16 y guarda los residuos de abajo hacia arriba.

Ej: $190 \div 16 = 11$ residuo 14 $\rightarrow BE_{16}$

4. Hexadecimal a Decimal

Multiplica cada dígito por 16^n según su posición.

Ej: $BE_{16} \rightarrow (11 \times 16^1) + (14 \times 16^0) = 176 + 14 = 190_{10}$

¿Qué son: bit, nibble y byte?

Unidad	Descripción
Bit	La unidad más pequeña de información: 0 o 1
Nibble	Conjunto de 4 bits. Ej: 1010
Byte	Conjunto de 8 bits. Equivale a 1 carácter. Ej: 11001010

Consulte:

¿Cuál es la estructura de un registro de memoria?

¿Cómo se crea un puntero en los lenguajes de programación?

¿Qué es un sistema de archivos y cuáles son los más utilizados?

¿Cómo se almacenan los datos en un disco duro?

Cuál es la estructura de un registro de memoria?

Un **registro de memoria** generalmente hace referencia a un **registro del procesador (CPU)**. Un registro es una pequeña cantidad de memoria rápida ubicada dentro del procesador, utilizada para almacenar datos temporalmente durante la ejecución de instrucciones.

Estructura típica de un registro:

- **Nombre o identificador:** Por ejemplo, AX, EAX, R1, etc.
- **Tamaño:** Puede ser de 8, 16, 32, o 64 bits, dependiendo de la arquitectura del procesador.
- **Propósito (según tipo de registro):**
 - **Registro de datos:** Para operaciones aritméticas/lógicas.
 - **Registro de direcciones:** Almacena direcciones de memoria.
 - **Registro de propósito general:** Uso flexible.
 - **Registro de control:** Como el contador de programa (PC) o el registro de estado.

Ejemplo en arquitectura x86:

EAX: Registro acumulador de 32 bits, usado comúnmente para operaciones aritméticas.

¿Cómo se crea un puntero en los lenguajes de programación?

Un **puntero** es una variable que almacena la dirección de memoria de otra variable.

En C/C++:

c

CopiarEditar

```
int x = 10;
```

```
int *ptr = &x; // ptr apunta a la dirección de x
```

Explicación:

- `*ptr`: Declara un puntero a un entero.
- `&x`: Obtiene la dirección de `x`.

En otros lenguajes (como Python o Java):

- No se usan punteros explícitamente, pero las referencias a objetos funcionan como punteros de alto nivel.

¿Qué es un sistema de archivos y cuáles son los más utilizados?

Un **sistema de archivos (File System)** es un componente del sistema operativo que organiza, almacena, recupera y administra los datos en un dispositivo de almacenamiento.

Funciones clave:

- Estructurar datos en archivos y carpetas.
- Administrar acceso, permisos y espacio.

Sistemas de archivos más utilizados:

- **NTFS (Windows)**: Avanzado, con soporte para permisos, compresión, journaling.
- **FAT32 / exFAT**: Compatibles con muchos dispositivos, pero limitados (por ejemplo, FAT32 no admite archivos > 4 GB).
- **ext4 (Linux)**: Estable, rápido y confiable.
- **APFS (Apple)**: Optimizado para SSDs, utilizado en macOS.

¿Cómo se almacenan los datos en un disco duro?

Los **discos duros (HDD)** almacenan datos de forma **magnética** sobre platos giratorios divididos en pistas y sectores.

Proceso de almacenamiento:

1. El disco contiene **platos giratorios** recubiertos con material magnético.
2. Una **cabeza de lectura/escritura** posicionada por un brazo accede a sectores específicos.

3. Los datos se escriben como **patrones de polarización magnética**.
4. El sistema de archivos organiza los datos en bloques o clústeres.

Estructura física:

- **Platos:** Superficies circulares donde se almacena la información.
- **Pistas:** Círculos concéntricos en cada plato.
- **Sectores:** Fragmentos de cada pista (típicamente 512 bytes o 4 KB).
- **Clusters:** Agrupación de sectores definida por el sistema de archivos.

En contraste, los SSDs almacenan datos electrónicamente en celdas de memoria flash, sin partes móviles.

Consulta:

¿Cuáles son los tipos de dato que existen en lenguajes de programación?

¿Cuáles son los tipos de dato que existen en las bases de datos?

¿Qué es un Collation en base de datos?

¿Cuáles son los tipos de dato que existen en lenguajes de programación?

Los **tipos de datos** en lenguajes de programación definen el tipo de valores que puede almacenar una variable. Se clasifican en varias categorías:

♦ *Tipos de datos primitivos (o básicos):*

- **Enteros (int, long, short):** Números enteros (ej. 5, -100).
- **Punto flotante (float, double):** Números con decimales (ej. 3.14, -0.001).
- **Carácter (char):** Un solo carácter (ej. 'a', 'Z').
- **Booleanos (bool):** Verdadero o falso (true, false).

♦ *Tipos de datos compuestos:*

- **Cadenas de texto (string):** Secuencia de caracteres (ej. "Hola mundo").
- **Arreglos (arrays):** Colección de datos del mismo tipo.
- **Estructuras (structs), registros u objetos:** Agrupan múltiples tipos de datos.

◊ *Tipos de datos abstractos (dependen del lenguaje):*

- Listas, conjuntos, mapas, diccionarios, colas, pilas, etc.

Ejemplo en C++:

```
cpp
CopiarEditar
int edad = 25;
float pi = 3.14;
bool activo = true;
char inicial = 'J';
string nombre = "Juan";
```

¿Cuáles son los tipos de dato que existen en las bases de datos?

Los **tipos de datos en bases de datos** definen la naturaleza de los datos almacenados en cada columna de una tabla.

◊ *Tipos numéricos:*

- INT, SMALLINT, BIGINT: Enteros.
- FLOAT, REAL, DOUBLE, DECIMAL, NUMERIC: Decimales (con o sin precisión definida).

◊ *Tipos de texto:*

- CHAR(n): Cadena de longitud fija.
- VARCHAR(n): Cadena de longitud variable.
- TEXT: Texto largo.

◊ *Tipos de fecha y hora:*

- DATE: Fecha (AAAA-MM-DD).
- TIME: Hora (HH:MM:SS).
- DATETIME / TIMESTAMP: Fecha y hora combinadas.

♦ *Tipos lógicos:*

- BOOLEAN o TINYINT (1) (en MySQL): Verdadero o falso.

♦ *Tipos binarios:*

- BLOB: Almacena datos binarios (imágenes, audio, etc.).

Nota: Los nombres exactos pueden variar según el motor (MySQL, PostgreSQL, SQL Server, Oracle, etc.).

¿Qué es un *Collation* en base de datos?

Un **Collation** es un conjunto de reglas que determina **cómo se comparan y ordenan los datos de texto** en una base de datos.

♦ *¿Qué aspectos define un collation?*

1. **Orden de los caracteres (sorting):** Si "a" va antes que "b", etc.
2. **Sensibilidad a mayúsculas y minúsculas (case sensitivity):** ¿"Hola" es igual a "hola"?
3. **Sensibilidad a acentos (accent sensitivity):** ¿"a" es igual a "á"?

♦ *Ejemplos de Collations:*

- utf8_general_ci: Insensible a mayúsculas y acentos.
- utf8_bin: Sensible a mayúsculas y acentos (compara según valores binarios).
- latin1_swedish_ci: Collation por defecto en algunas instalaciones antiguas de MySQL.

ci = *case-insensitive*

cs = *case-sensitive*

ai = *accent-insensitive*